



CODEFEST 2026:
THE ACLC MANDAUE HACKATHON
PRE-HACKTOBERFEST EDITION



git &



GitHub

WORKSHOP

From Version Control to Collaboration



Learn Git



Manage Repositories



Create Branches



Push to GitHub



Collaborate





FUNDAMENTALS

Understand the basics. Build with confidence.





GIT & GITHUB

What is the Difference Between ?





GIT & GITHUB

What is the Difference Between



```
Dev@aclc: ~/workshop $ gitHub --version
GitHub CLI version 2.75.0

Github is the platform for hosting repositories and collaborating
(online)
```

```
Dev@aclc: ~/workshop $ git --version
git version 2.51.0

Git is the version control system (runs on your
computer)
```



GIT & GITHUB

GLOBAL CONFIGURATION

< / > Bash

There are two types of configurations

GLOBAL	LOCAL
<pre>\$ git config --global user.name "Your Name" \$ git config --global user.email "your@email.com"</pre>	<pre>\$ git config --local user.name "Your Name" \$ git config --local user.email "your@email.com"</pre>



GIT & GITHUB

GLOBAL CONFIGURATION COMMANDS

```
< / > Bash

$ git config --global user.name "Your Name"           Set / change username           Global or Local
$ git config --global user.email "your@email.com"     Set / change email             Global or Local
$ git config --global --list                          View all global configurations  Global or Local
$ git config --global user.name                      Check specific configuration    Global or Local
$ git config --global --unset user.name              Remove configuration            Global or Local
```



GIT & GITHUB

These are the quick setup instructions provided when creating a new repository on GitHub.

< / > Bash

```
$ git init
```

```
Output: Initialized empty Git repository in .../my-project/.git/
```

```
$ git add README.md (or .) all files
```

```
$ git commit -m "Message"
```

```
$ git branch -M main
```

```
$ git remote add origin https://github.com/example/git&github.git
```

```
$ git push -u origin main
```

Or push an existing local repository to a new GitHub repository

```
$ git remote add origin https://github.com/example/git&github.git
```

```
$ git branch -M main
```

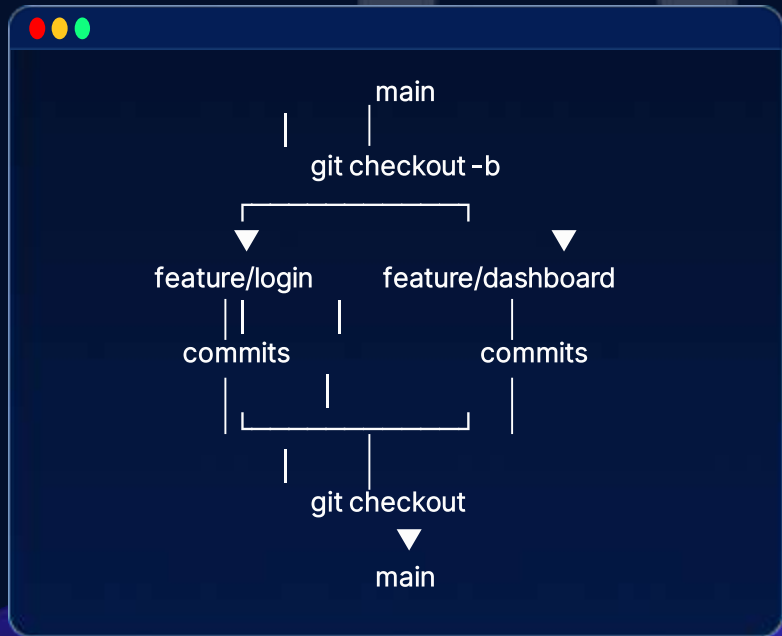
```
$ git push -u origin main
```



GIT & GITHUB

Why is it important to use branches and git checkout?

Branches separate development work, while git checkout lets you switch between branches.

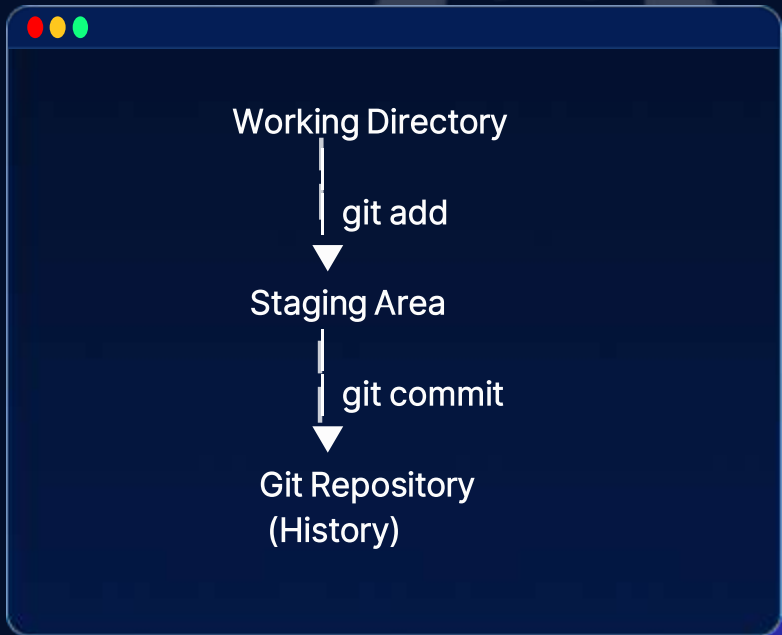




GIT & GITHUB

Why it is important to use add and commit?

\$ git add and \$ git commit are important because they let you select and save your changes into Git's version history.





GIT & GITHUB

How does git fetch work?
How does git merge work?
How does git pull work?

git fetch downloads remote changes, **git merge** combines changes between branches, and **git pull** fetches and merges remote changes into the current branch.

Git Fetch



Download



Git Merge



Combine



Git Pull



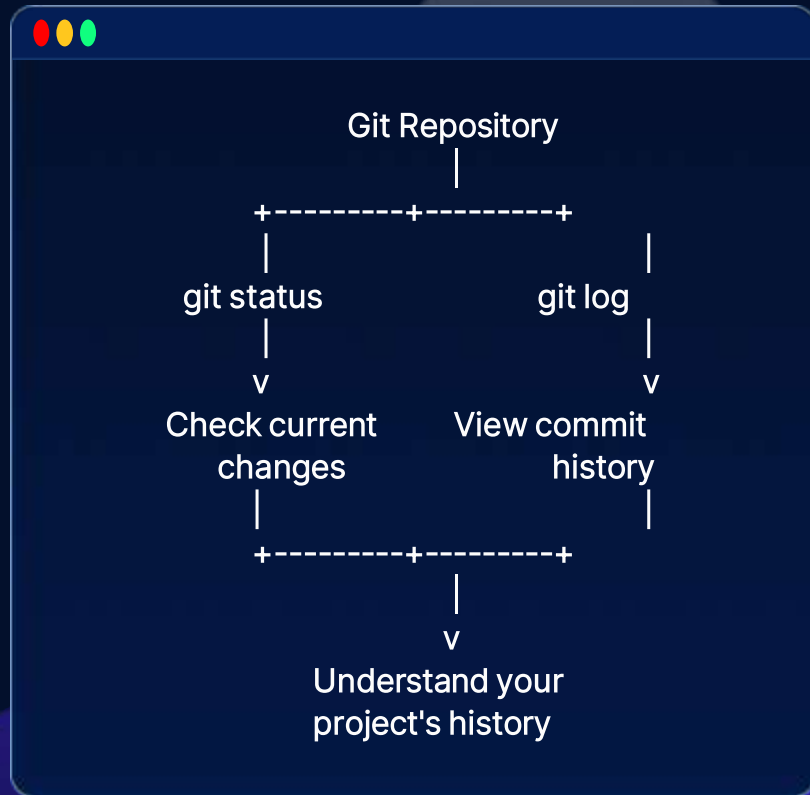
Fetch + Merge



GIT & GITHUB

Why is it important to check logs and status?

They help you understand the current state of your repository and track previous changes.





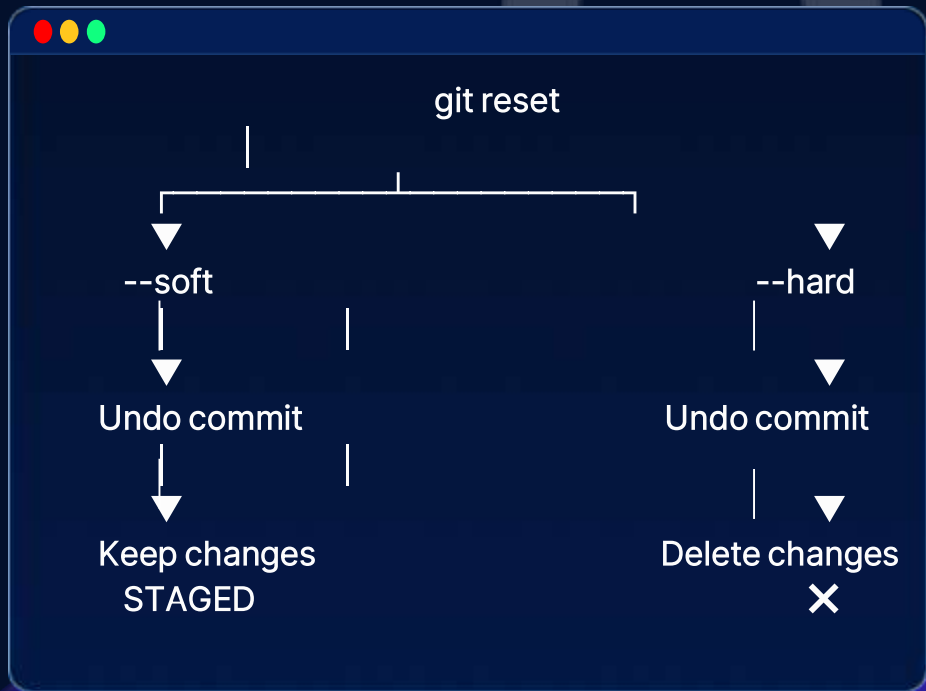
GIT & GITHUB

What is difference between \$ git reset --soft or --hard
Why is it important to know git reset --soft and --hard?

It is important because they let you undo commits safely or completely, depending on what you want to keep.

--soft → Undo commit + Keep changes staged

--hard → Undo commit + Delete changes





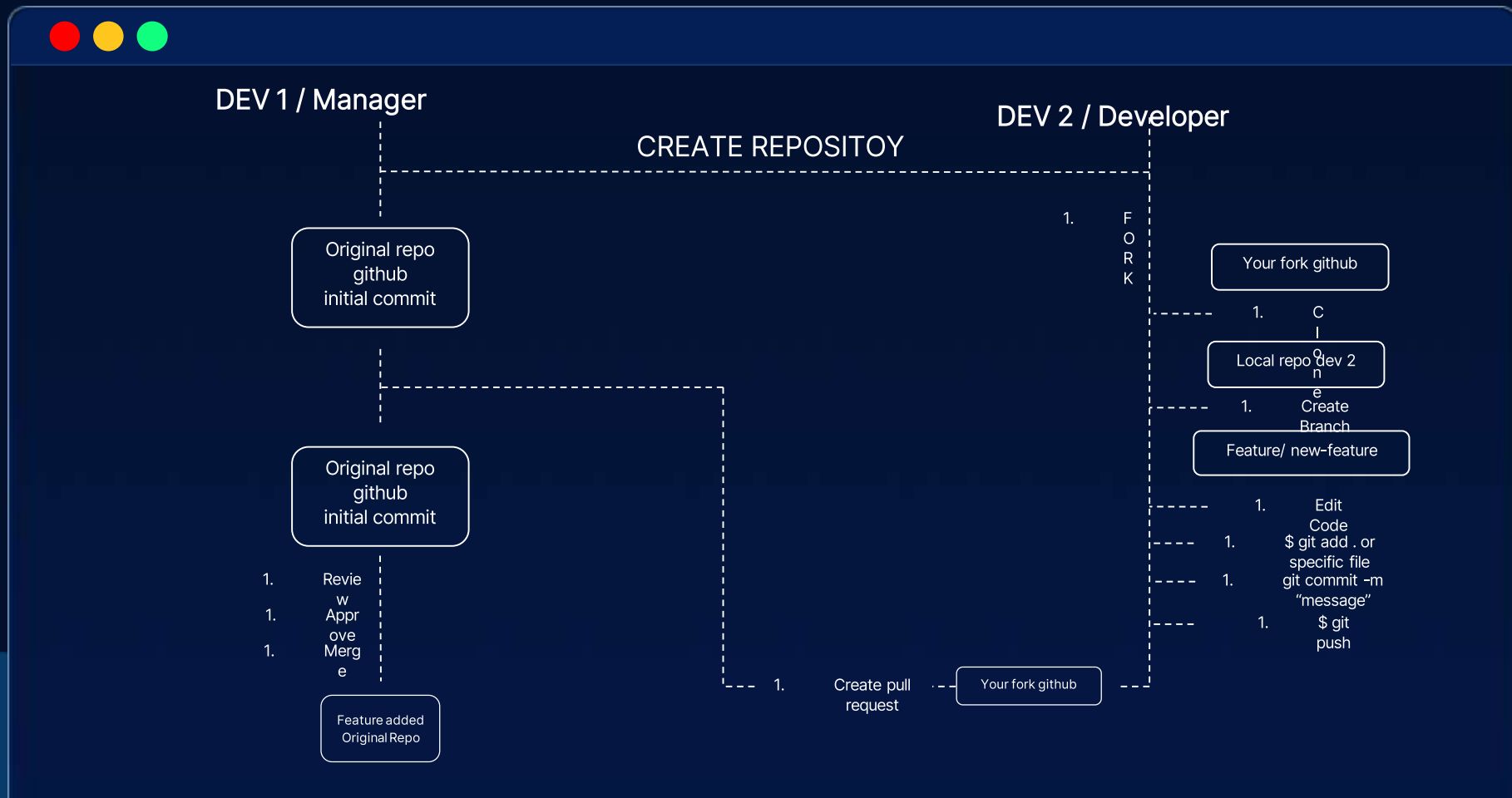
Hands-on Activity



GIT & GITHUB

HANDS-ON ACTIVITY SCENARIO

1. Manager creates a public repository with an initial commit.
2. Developer forks the repository and clones the forked repository to their local machine.
3. Developer creates a new branch for the feature.
4. Developer works on the feature and makes the necessary code changes.
5. Developer stages the changes using git add.
6. Developer commits the changes using git commit.
7. Developer pushes the feature branch to their fork using git push.
8. Developer creates a Pull Request (PR) from their feature branch to the manager's original repository.
9. Manager reviews the Pull Request and requests changes if necessary.
10. Developer makes additional changes, commits, and pushes again if changes are requested.
11. Manager approves the Pull Request.
12. Manager merges the Pull Request into the original repository.
13. Developer can sync/update their fork with the original repository for future work.





THANK YOU ...